

# Calibrated regression by binning the target: a probabilistic model compared with gradient-boosted trees and a plain network

Jordan Elridge

16 July 2026

## Summary

---

This note compares six regression models on five one-dimensional synthetic targets. Two of the models are versions of the same idea — split the target range into bins, learn to weight the bins from the input, and refine within each bin — one with a fixed number of bins, one that learns how many bins to use. The other four are standard regressors: three gradient-boosted tree systems and a plain feed-forward network. Every model is scored on both point accuracy and on the quality of the uncertainty it reports.

The headline is about *calibration* — whether the interval a model reports around its prediction contains the true value as often as it claims. On that measure the binning model, and especially its adaptive-bin version, is the most reliable of the six: on the targets with genuine input-varying noise its ninety-percent intervals contain the truth close to ninety percent of the time. The tree systems achieve lower squared error on several of the targets, but on those same targets their intervals are systematically too narrow — they cover the truth noticeably less often than ninety percent. (The one exception is a deliberately two-peaked target, where a single interval is the wrong shape for every model and nearly all of them over-cover; it is discussed in its place.) This is the central caution of the note: a narrow interval is not a better interval. The honest score for an interval is how often it actually covers the truth, and only among intervals that cover can width be compared.

## 1. What is being compared, and why it matters

---

A regression model that only returns a single number cannot say how sure it is. For decisions that depend on the tail — a safety margin, a risk budget, a trigger threshold — the useful output is not a point but a *predictive distribution*: for each input  $x$ , a full statement of which target values are plausible and with what probability. Two things then need checking. First, is the centre of that distribution accurate (point accuracy)? Second, is its spread honest (calibration) — when the model says “ninety percent chance the truth lies in this interval”, does the truth land there ninety percent of the time?

The six models below all produce a predictive distribution, but by very different routes, and the note’s purpose is to see which route yields *calibrated* spread without giving up too much point accuracy.

## 2. The binning model

---

## 2.1 The idea: turn regression into weighting plus refinement

Rather than predict the target directly, the model first splits the range of the target into  $K$  contiguous bins — a fixed whole number, whose choice is the subject of Section 2.2 — indexed by  $k = 1, \dots, K$ , each holding an equal share of the training targets (the bin edges are the percentiles of the training targets). A **classifier** — a small network reading the input  $x$  — outputs a score  $\ell_k(x)$  for each bin, turned into a probability by the softmax function,

$$\pi_k(x) = \frac{\exp(\ell_k(x))}{\sum_{j=1}^K \exp(\ell_j(x))}, \quad k = 1, \dots, K, \quad \sum_{k=1}^K \pi_k(x) = 1. \quad (1)$$

So  $\pi_k(x)$  is the model's belief that the target for input  $x$  falls in bin  $k$ . This is a genuine classifier over  $K$  classes; it is *not* a fixed Gaussian mixture, because the class weights  $\pi_k(x)$  are computed afresh from each input.

Each bin also carries a small **regression head** that outputs, again as a function of  $x$ , a local mean  $\mu_k(x)$  and a local variance  $\sigma_k^2(x)$  (the head emits  $\log \sigma_k^2$  so the variance is guaranteed positive). The head refines the coarse bin into a continuous local prediction.

The single point prediction is the weighted average of the bin means,

$$\hat{y}(x) = \sum_{k=1}^K \pi_k(x) \mu_k(x), \quad (2)$$

and the predictive variance combines two sources of spread through the **law of total variance** — the identity that the variance of a quantity equals the average of its within-group variances plus the variance of its group means (derived in full in Appendix A):

$$\hat{\sigma}^2(x) = \underbrace{\sum_{k=1}^K \pi_k(x) \sigma_k^2(x)}_{\text{within-bin noise}} + \underbrace{\sum_{k=1}^K \pi_k(x) (\mu_k(x) - \hat{y}(x))^2}_{\text{disagreement between bins}}. \quad (3)$$

The first term is the noise the model expects *inside* whichever bin the input lands in. The second term grows when the model spreads its weight across bins whose means disagree — that is, when the model is genuinely unsure which part of the range the target belongs to. When all the weight sits on one bin, or all bin means agree, the second term vanishes and only the within-bin noise remains. For scoring, the predictive distribution is summarised as a Gaussian with this mean and variance,  $\mathcal{N}(\hat{y}(x), \hat{\sigma}^2(x))$ .

Because the variances  $\sigma_k^2(x)$  are *learned* and vary with the input, this model can report a spread that changes from one input to another — wide where the data are noisy, narrow where they are clean. That property is what the calibration results below turn on.

## 2.2 Fixed number of bins

In the first version the number of bins  $K$  is fixed before training. It is not guessed: for each target,  $K$  is chosen from a small grid  $\{5, 8, 10, 12\}$  — the value with the lowest negative log-likelihood on a separate, held-out draw

of data — and then frozen. The chosen values differ by target (Table 1) and the reason they differ is itself a finding, taken up in the limitations (Section 7): too few bins cannot resolve a noisy target.

## 2.3 Adaptive number of bins

The second version keeps the same per-bin machinery but adds a small **gating network** that reads the input  $x$  and chooses, softly and per input, *how finely to bin*. Concretely it weights a bank of candidate resolutions — a classifier using two bins, one using three, and so on up to a maximum  $K_{\max}$  (here twelve) — together with one “bypass” option that skips binning and predicts the target directly. Each binning candidate forms its own predictive mean  $\mu_j(x)$  and variance  $\sigma_j^2(x)$  by the law of total variance of Section 2.1; the bypass predicts a mean and variance directly, from a single Gaussian head, without binning. Writing  $q_j(x)$  for the gate’s weight on candidate  $j$ , the model’s mean and variance for input  $x$  are combined by the *same* law of total variance one level up — over the gate rather than over one classifier’s bins:

$$\hat{y}(x) = \sum_j q_j(x) \mu_j(x), \quad \hat{\sigma}^2(x) = \sum_j q_j(x) \sigma_j^2(x) + \sum_j q_j(x) (\mu_j(x) - \hat{y}(x))^2. \quad (4)$$

So a simple input can rest on a coarse resolution or the bypass while a hard one draws on a finer one.

It is trained with an extra penalty — a Kullback–Leibler term, Appendix B — that keeps the gate *honest rather than opinionated*: it carries no built-in preference for any particular resolution (it pulls the bypass rate toward an even one-half and the weighting across bin counts toward uniform), so the gate concentrates on a resolution only where the data give it reason to. This is treated throughout as a **separate model**, not a setting of the first: it has its own parameters and its own behaviour, and — as the results show — it is generally the stronger of the two. In the tables and figures the two versions appear as *Binning (fixed)* and *Binning (adaptive)*.

## 3. What the model is trained to minimise

---

The two binning models are trained to minimise the **negative log-likelihood** of the true target under their predictive Gaussian — mean and spread learned together. The four baselines of Section 4 instead fit only a mean and attach their spread after the fact. For a single example with target  $y$ , predicted mean  $\hat{y}$  and predicted variance  $\hat{\sigma}^2$ , that is

$$\mathcal{L}_{\text{NLL}} = \frac{1}{2} \log(2\pi\hat{\sigma}^2) + \frac{(y - \hat{y})^2}{2\hat{\sigma}^2} \quad (5)$$

(here  $\pi$  inside  $2\pi$  is the circle constant 3.14159..., not a bin weight; the derivation from the Gaussian density is in Appendix A). The fixed-bin model is trained on this likelihood alone; the adaptive model adds one further term, the gating penalty of Section 2.3 (Appendix B), which shapes how the gate spreads its weight across resolutions. In both models the softmax bin weights  $\pi_k(x)$  are learned entirely through this likelihood — there is no separate classification target — so the log-likelihood is the term that does the work.

Why minimise this rather than plain squared error? Squared error scores only the mean; it is completely indifferent to the spread the model reports, so it gives no reason to report an honest variance. The negative log-likelihood scores the whole distribution. It is a **strictly proper scoring rule** (Gneiting and Raftery 2007): a score whose expected value is optimised *only* when the reported distribution matches the true conditional

distribution of the target — no other reported distribution can do better in expectation. Read term by term, it rewards exactly the behaviour we want. The second term punishes squared error, but *divided by* the reported variance, so a model cannot lower its loss by shrinking its intervals unless it is genuinely that accurate; the first term punishes over-wide intervals, so it cannot hide by inflating them either. A model that is confidently wrong — small  $\hat{\sigma}^2$ , large error — pays a very large penalty. This is why the log-likelihood, and the calibration it induces, is the primary lens of this note.

## 4. The four baselines

---

**Three gradient-boosted tree systems** — XGBoost (Chen and Guestrin 2016), LightGBM (Ke et al. 2017) and CatBoost (Prokhorenkova et al. 2018). Gradient boosting builds an additive ensemble of small regression trees, each new tree fitted to reduce the error left by the ones before it. A plain boosted ensemble returns only a point; to give it a predictive spread on equal footing, all three use the same post-hoc estimate — the model’s training predictions are sorted into ten equal-count bins (by their percentiles), and the spread reported for a new prediction is the standard deviation of the training residuals that fell in the bin its prediction lands in (a **binned residual spread**). This varies from one prediction to another, but it is measured after the fact from residuals rather than learned as part of the fit. The three tree systems use this identical uncertainty estimate so they can be compared like for like.

**A plain feed-forward network**, trained to predict the mean by squared error, then assigned a single *constant* spread — the standard deviation of its training residuals, applied to every input. It therefore cannot widen its intervals where the data are noisier; it is the natural control for “what if the spread does not adapt to the input at all”.

## 5. Targets and protocol

---

The five targets (all one input, one output; sample sizes chosen per target) split into a **home-turf group** built to have interesting, input-dependent uncertainty, and a **standard group** of ordinary regression shapes:

- **Heteroscedastic** (home turf): a smooth curve  $y = 2 \sin x + \frac{1}{2}x$  with Gaussian noise whose standard deviation *grows with the input*,  $0.1 + 0.4|x|$  — so the honest spread is small near the origin and large at the edges.
- **Multimodal** (home turf): for each input the target is one of two parallel branches,  $y = x + 1.5$  or  $y = x - 1.5$ , chosen by a coin flip, with a little noise. The true conditional distribution has *two* separated peaks, so a single Gaussian summary can at best straddle them.
- **Three-region** (home turf): the input line is split into three equal stretches — an easy sloping line, a hard oscillation, and a *second* easy sloping line whose noise is ten times larger than the other two. It stresses whether a model behaves sensibly where the signal is simple but the noise is high.
- **Piecewise** (standard): a line with a kink at the origin,  $y = \frac{1}{2}x$  on the left and  $y = \frac{1}{2}x + \sin(4\pi x)$  on the right, so the right half carries a fast oscillation on top of the trend, with constant Gaussian noise of standard deviation 0.2.
- **Exponential** (standard):  $y = e^x$  over a range that makes the target span from about 0.05 to about 20, with constant additive noise — a wide dynamic range.

**Protocol.** Every result is the average over five independent draws of the target (true replicates, each drawn fresh, not re-splits of one draw). Sample sizes are per target — heteroscedastic and multimodal use one

thousand points, three-region nine hundred, piecewise and exponential eight hundred — and each draw is split seventy–thirty into a training and a test part, with the test part scored exactly once. Data roles are kept separate: the choice of bin count uses its own dedicated draw at a seed outside the five. Every ninety-percent interval, for every model, is formed the same way — the predictive mean plus or minus 1.645 predictive standard deviations, the central ninety percent of a Gaussian — so coverage and width are compared like for like. Iterative training is watched for early stopping: the networks are stopped by held-out score and the tree ensembles by their boosting budget, and any run whose best score arrived only as its budget ran out is flagged and kept out of the headline claims (in practice this caught only two of the tree systems on the piecewise target — see Section 7).

**The seven scores**, for each model on each target:

- **Mean squared error** — the average squared gap between the point prediction and the truth. Lower is better. Scores the centre only.
- **Negative log-likelihood** — the score of Section 3, averaged over the test set. Lower is better. Scores the whole predictive distribution.
- **Calibration error** — built from the *probability integral transform*: for each test point, compute the predictive probability of a value at or below the truth; if the model is calibrated these numbers are spread evenly between zero and one. The calibration error is the average gap, across ten equally spaced probability levels, between how often the model predicts the truth should fall below a level and how often it actually does (Appendix C). Lower is better.
- **Ninety-percent coverage** (written *Coverage@90* in the tables) — the fraction of test points whose true value falls inside the model’s ninety-percent interval. The target is 0.90; below that is *under-coverage* (intervals too narrow), above is over-coverage (too wide).
- **Ninety-percent interval width** — the average width of those intervals. Lower is better *only among models that actually reach the coverage target*; a narrow interval that under-covers is not an achievement.
- **Interval score** at ninety percent — a single proper score for an interval (Gneiting and Raftery 2007): it charges the interval’s width, plus a penalty that grows with the size of the miss whenever the truth falls outside. Lower is better. Unlike width on its own, it cannot be lowered by shrinking an interval that then fails to cover — the two ways an interval can be wrong, too wide and too narrow, are penalised together in one number (its formula is in Appendix C). It is the score that most directly captures this note’s argument.
- **Distribution score** — the continuous ranked probability score, a proper score for the *whole* predictive distribution in the units of the target (Gneiting and Raftery 2007): the integrated squared gap between the predicted cumulative distribution and the step that rises at the true value (closed form in Appendix C). Lower is better. It stays finite and well-behaved even when a model is confidently wrong, where the log-likelihood blows up, so it is the robust cross-check on the log-likelihood.

The bin count chosen for the fixed-bin model, per target, is given in Table 1; the same values are used for the fixed-bin rows throughout.

**Table 1. Bin count selected per target, on a separate held-out draw.**

| Target          | Bins used |
|-----------------|-----------|
| Heteroscedastic | 8         |
| Multimodal      | 10        |
| Three-region    | 12        |
| Piecewise       | 10        |

| Target      | Bins used |
|-------------|-----------|
| Exponential | 12        |

## 6. Results

### 6.1 Home-turf targets

Tables 2 and 3 give the accuracy and the calibration scores on the three home-turf targets; Figure 1 shows the coverage picture those numbers add up to, and Figure 2 the reliability of each model’s whole predictive distribution. Every value is a mean over the five draws, and bold marks the lowest mean in each accuracy column (and the coverage nearest ninety percent), or both values when they tie at the shown precision — a marker, not a significance test. The seed-to-seed standard deviations are small against the calibration gaps that carry the argument — a few thousandths on the calibration-error and coverage scores — but on squared error they run from about a hundredth on the sharpest targets to a fifth on the noisiest, so an accuracy lead smaller than that should be read as a tie rather than a win. (The lone exception, the fixed-bin model’s instability on the exponential target, is taken up in Section 7.)

**Table 2. Home-turf targets — accuracy (lower is better; best per column in bold).**

| Target          | Model              | Sq. error    | Neg. log-lik. | Distrib. score |
|-----------------|--------------------|--------------|---------------|----------------|
| Heteroscedastic | Binning (fixed)    | 1.989        | 1.526         | 0.727          |
|                 | Binning (adaptive) | <b>1.624</b> | <b>1.386</b>  | <b>0.648</b>   |
|                 | XGBoost            | 1.845        | 1.822         | 0.732          |
|                 | LightGBM           | 1.685        | 1.658         | 0.695          |
|                 | CatBoost           | 1.651        | 1.654         | 0.687          |
|                 | Plain network      | 1.630        | 1.669         | 0.693          |
| Multimodal      | Binning (fixed)    | 2.337        | 1.846         | 0.917          |
|                 | Binning (adaptive) | <b>2.290</b> | <b>1.834</b>  | <b>0.909</b>   |
|                 | XGBoost            | 2.510        | 1.943         | 0.971          |
|                 | LightGBM           | 2.373        | 1.863         | 0.932          |
|                 | CatBoost           | 2.301        | 1.837         | 0.915          |
|                 | Plain network      | 2.323        | 1.841         | 0.914          |
| Three-region    | Binning (fixed)    | 0.145        | 0.013         | 0.185          |
|                 | Binning (adaptive) | 0.140        | -0.112        | 0.175          |
|                 | XGBoost            | 0.106        | 0.257         | 0.149          |
|                 | LightGBM           | <b>0.100</b> | <b>-0.212</b> | <b>0.140</b>   |
|                 | CatBoost           | <b>0.100</b> | -0.155        | 0.144          |
|                 | Plain network      | 0.118        | 0.348         | 0.182          |

**Table 3. Home-turf targets — calibration (coverage nearest 0.90 in bold; interval score and calibration error lower is better; width is not bolded — narrow only helps if it covers).**

| Target          | Model              | Coverage@90  | Width@90 | Interval score | Calib. err   |
|-----------------|--------------------|--------------|----------|----------------|--------------|
| Heteroscedastic | Binning (fixed)    | <b>0.894</b> | 4.115    | 5.495          | 0.034        |
|                 | Binning (adaptive) | 0.882        | 3.724    | <b>4.855</b>   | 0.028        |
|                 | XGBoost            | 0.809        | 3.209    | 6.729          | 0.028        |
|                 | LightGBM           | 0.831        | 3.563    | 5.911          | <b>0.026</b> |
|                 | CatBoost           | 0.849        | 3.610    | 5.832          | 0.028        |
|                 | Plain network      | 0.869        | 3.943    | 6.009          | 0.038        |
| Multimodal      | Binning (fixed)    | 0.999        | 5.038    | 5.038          | 0.128        |
|                 | Binning (adaptive) | 1.000        | 4.911    | 4.911          | 0.129        |
|                 | XGBoost            | <b>0.875</b> | 4.195    | 5.055          | 0.124        |
|                 | LightGBM           | 0.979        | 4.614    | <b>4.686</b>   | <b>0.117</b> |
|                 | CatBoost           | 1.000        | 4.830    | 4.830          | 0.137        |
|                 | Plain network      | 1.000        | 4.972    | 4.972          | 0.125        |
| Three-region    | Binning (fixed)    | 0.877        | 1.021    | 1.310          | 0.066        |
|                 | Binning (adaptive) | <b>0.917</b> | 1.062    | <b>1.264</b>   | 0.066        |
|                 | XGBoost            | 0.772        | 0.653    | 1.459          | 0.094        |
|                 | LightGBM           | 0.823        | 0.714    | 1.294          | <b>0.048</b> |
|                 | CatBoost           | 0.834        | 0.754    | 1.291          | 0.057        |
|                 | Plain network      | 0.876        | 1.088    | 1.634          | 0.077        |

On the **heteroscedastic** target — the one built for input-dependent spread — the binning model is the calibration winner. Its ninety-percent intervals cover the truth 0.894 (fixed bins) and 0.882 (adaptive bins) of the time, against a target of 0.90; the three tree systems cover only 0.809, 0.831 and 0.849, and the constant-spread network 0.869. The tree intervals are *narrower* (width about 3.2–3.6 versus 3.7–4.1 for the binning model), which is exactly the trap: they are narrower because they under-cover, not because they are better. The adaptive-bin model also has the lowest negative log-likelihood on this target (1.386, against 1.65–1.82 for the trees), confirming that its whole predictive distribution, not just its coverage, is the most honest.

On the **multimodal** target every model is squeezed by the same limit: a single Gaussian summary cannot represent two separated peaks, so every model’s calibration error is high (roughly 0.12–0.14) and its intervals are wide enough to swallow both branches — coverage at or near one for most models, with XGBoost the exception at 0.875. (For a model that covers almost every point the interval score in Table 3 equals its width exactly — the miss penalty of Appendix C never fires — which is why those two columns coincide on this target.) The binning model’s negative log-likelihood is still the lowest of the six, but this target is really a reminder that a Gaussian predictive summary — the common ground on which all six are scored — is the wrong shape here for everyone.

On the **three-region** target the split between accuracy and calibration is at its sharpest. The tree systems win squared error (0.100–0.106 versus 0.14–0.15 for the binning model), because the underlying signal is mostly simple lines that trees fit tightly. But the adaptive-bin model has the best coverage of any model (0.917, essentially on target) while the trees under-cover (0.772–0.834), and its negative log-likelihood falls within the trees’ range (behind the two best, ahead of the third). Where the noise is high but the signal is simple, the

learned, input-varying spread of the binning model reports the uncertainty honestly; the trees fit the mean well and then under-state how uncertain they are.

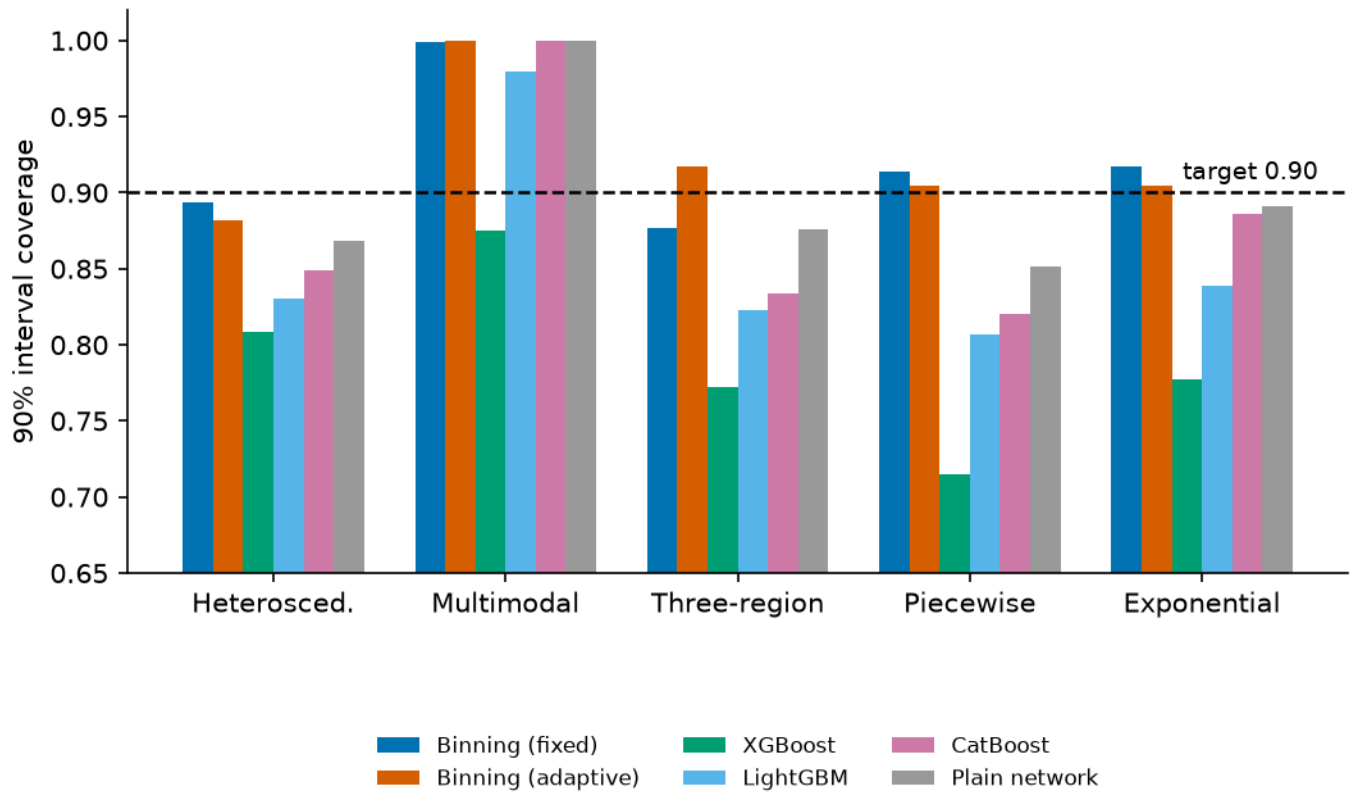


Figure 1: Coverage of the ninety-percent intervals, by model and target. The dashed line is the nominal ninety percent; a bar below it means the model’s intervals contain the truth less often than they claim. The tree systems sit below the line on every target except the two-peaked (multimodal) one, where a single interval is the wrong shape and most models over-cover instead — XGBoost is the exception, still just below the line.



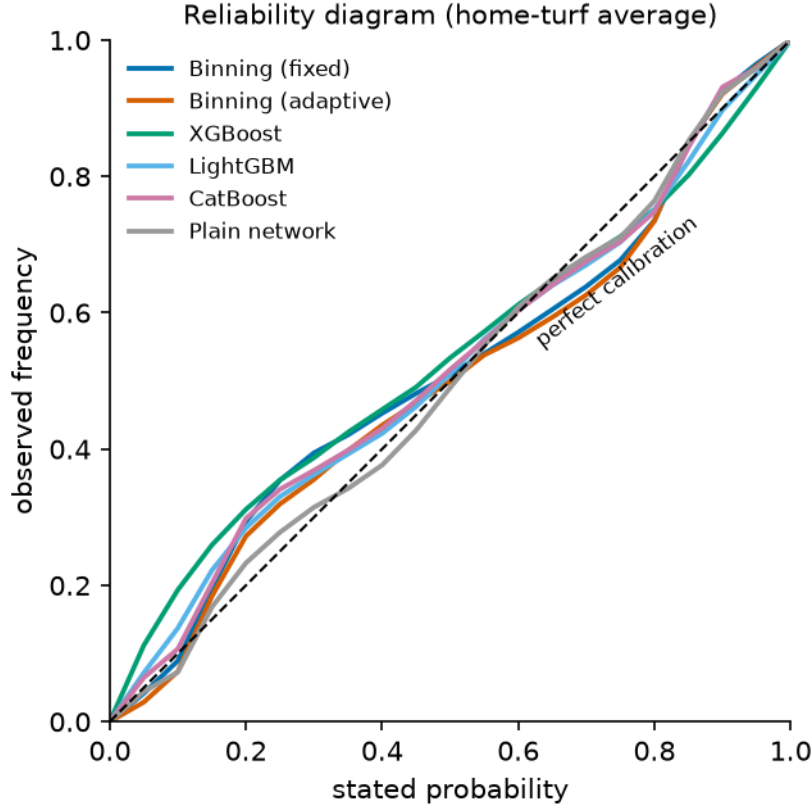


Figure 2: Reliability of the predictive distributions, averaged over the home-turf targets: for each stated probability (horizontal axis), the fraction of test points whose true value actually falls at or below the model’s quantile at that probability (vertical axis). A curve on the diagonal is perfectly calibrated. A curve that runs above the diagonal at low stated probabilities and below it at high ones is overconfident — its predictive distributions are too narrow — which is the same failure the coverage numbers show for the tree systems; the tree curves deviate most. This is the whole-distribution companion to the ninety-percent coverage in Figure 1.

## 6.2 Standard targets

Tables 4 and 5 give the standard group. Here the point-accuracy advantage of the trees is largest, and it is real: on **piecewise** the trees reach a squared error of 0.081–0.124 against 0.30 for the binning model, and on **exponential** the best tree reaches 0.253. The binning model, which represents the target through a fixed set of bins, pays for the sharp kink and the fast oscillation of the piecewise target with a coarser point fit.

**Table 4. Standard targets — accuracy (lower is better; best per column in bold; † flags a non-converged cell, explained in the note after Table 5).**

| Target    | Model              | Sq. error    | Neg. log-lik. | Distrib. score |
|-----------|--------------------|--------------|---------------|----------------|
| Piecewise | Binning (fixed)    | 0.304        | 0.570         | 0.284          |
|           | Binning (adaptive) | 0.295        | 0.517         | 0.275          |
|           | XGBoost            | <b>0.081</b> | 0.374         | <b>0.160</b>   |
|           | LightGBM †         | 0.124        | 0.330         | 0.191          |

| Target      | Model              | Sq. error    | Neg. log-lik. | Distrib. score |
|-------------|--------------------|--------------|---------------|----------------|
| Exponential | CatBoost †         | 0.090        | <b>0.208</b>  | 0.166          |
|             | Plain network      | 0.341        | 0.887         | 0.323          |
|             | Binning (fixed)    | 1.478        | 1.029         | 0.481          |
|             | Binning (adaptive) | 0.267        | 0.761         | 0.290          |
|             | XGBoost            | 0.286        | 0.917         | 0.307          |
|             | LightGBM           | 0.369        | 0.886         | 0.328          |
|             | CatBoost           | <b>0.253</b> | <b>0.738</b>  | <b>0.283</b>   |
|             | Plain network      | 0.328        | 0.849         | 0.318          |

**Table 5. Standard targets — calibration (coverage nearest 0.90 in bold; interval score and calibration error lower is better; width is not bolded).**

| Target      | Model              | Coverage@90  | Width@90 | Interval score | Calib. err   |
|-------------|--------------------|--------------|----------|----------------|--------------|
| Piecewise   | Binning (fixed)    | 0.914        | 1.686    | 1.868          | 0.058        |
|             | Binning (adaptive) | <b>0.905</b> | 1.521    | 1.771          | 0.044        |
|             | XGBoost            | 0.715        | 0.592    | 1.398          | 0.065        |
|             | LightGBM †         | 0.807        | 0.879    | 1.446          | 0.044        |
|             | CatBoost †         | 0.820        | 0.766    | <b>1.286</b>   | <b>0.038</b> |
|             | Plain network      | 0.852        | 1.790    | 2.651          | 0.066        |
| Exponential | Binning (fixed)    | 0.917        | 3.051    | 3.418          | 0.049        |
|             | Binning (adaptive) | <b>0.904</b> | 1.752    | 2.172          | 0.040        |
|             | XGBoost            | 0.777        | 1.316    | 2.473          | 0.056        |
|             | LightGBM           | 0.839        | 1.699    | 2.594          | 0.029        |
|             | CatBoost           | 0.886        | 1.594    | <b>2.127</b>   | <b>0.025</b> |
|             | Plain network      | 0.891        | 1.776    | 2.398          | 0.094        |

† On the piecewise target, four of the five LightGBM runs and three of the five CatBoost runs reached their best score only at the end of their training budget rather than settling before it (Section 7). Their cell averages include those runs and should be read as a lower bound on error, not a converged figure.

But the coverage story is unchanged, and it is the point of the note. On piecewise the binning model covers at 0.914 and 0.905 — on target — while the trees cover at 0.715, 0.807 and 0.820; the worst-covering tree misses its stated ninety percent by nearly nineteen points. On exponential the binning model again covers near target (0.917, 0.904) and the trees under-cover (0.777–0.886). One honest qualification: on these two targets the trees are accurate enough that their tighter intervals also win the *interval score* (Table 5) — the score that otherwise penalises under-coverage — so here the binning model’s advantage is specifically about reaching the coverage target, not about the interval score. On the two noisy home-turf targets — heteroscedastic and three-region — the interval score does favour the binning model (Table 3); on the two-peaked target no model is well-calibrated and a tree edges it there too. Figure 3 places every model on the two axes that matter together — how often its intervals cover, against how wide they are — and the pattern is a clean separation: the tree systems sit low and narrow (under-covering with tight intervals), the binning model sits on the coverage line at moderate width, and the constant-spread network sits off to the side, unable to place its fixed-width intervals well on any target with input-dependent noise.

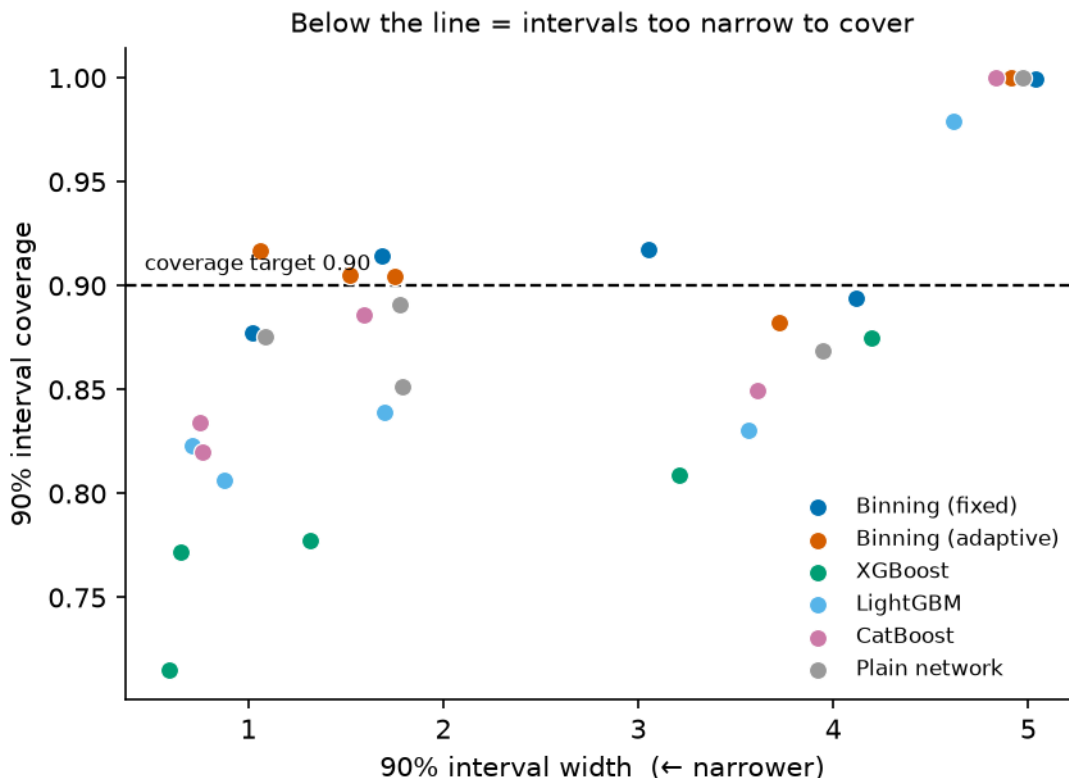


Figure 3: Coverage against interval width, one point per model per target. The horizontal dashed line is the nominal ninety percent. Points below it under-cover; the tree systems cluster there at small width (narrow intervals that miss), while the binning model’s points sit on or above the line — it reaches the coverage target, and over-covers only on the two-peaked target (the cluster near coverage one at upper right, where nearly every model’s intervals are too wide — XGBoost is the exception, sitting just below the line).

## 7. Limitations

**Bins must be resolved to the target’s noise, and too few fail.** The fixed-bin counts chosen per target (Table 1) are not cosmetic: they are 8, 10, 12, 10, 12 — never the smallest option on the grid. Too few bins cannot resolve a noisy target’s spread: the score is then dominated by the coarseness of the bins rather than by any mis-estimated variance, and only adding bins recovers it. This is why the bin count is selected per target rather than fixed globally, and it is a genuine cost of the method — the resolution has to be matched to the data.

**The fixed-bin model can be unstable on wide-range targets.** On the exponential target the fixed-bin model’s squared error is not only worse than the trees’ but erratic across the five draws: a mean of 1.478 with a standard deviation of 0.93 (one draw alone reaches 3.25), almost as large as the mean itself. The adaptive-bin model does not show this — 0.267 with a standard deviation of 0.03 — which is a further reason to prefer it: letting the model choose its bin usage stabilises the wide-range case.

**A Gaussian predictive summary cannot represent genuinely multi-peaked targets.** On the multimodal target no model scored here can be well-calibrated, because all are summarised and scored as a single Gaussian. The binning model carries more information internally (its bin weights can be two-peaked), but that information

is discarded by the collapse to a single mean and variance used for scoring. Reporting the full multi-peaked distribution is out of this note’s scope.

**Two tree cells did not fully converge.** On the piecewise target, three of the five CatBoost runs and four of the five LightGBM runs reached their best score only at the end of their training budget rather than settling before it, so those cells are flagged and excluded from the convergence-sensitive claims above. Their point accuracy is reported for completeness but should be read as a lower bound on error, not a converged figure.

## 8. When to use which

- **When the decision depends on calibrated intervals** — a coverage guarantee, a risk margin, a threshold with a controlled false-alarm rate — use the binning model, and prefer the **adaptive-bin** version. It is the only model here whose ninety-percent intervals actually cover near ninety percent on the targets with input-varying noise, and its negative log-likelihood is the lowest or near-lowest on those same targets.
- **When only the point matters** on a smooth target, and the reported spread will be ignored, a gradient-boosted tree system gives the lowest squared error at low cost. But do not read its intervals as calibrated: on the noisy targets here they under-cover, by up to nearly nineteen points (on the two-peaked target two of the three tree systems, like most models, over-cover instead).
- **Do not judge intervals by width.** Across the board the narrowest intervals belong to the models that cover least. Compare widths only after checking that coverage is met.
- **Match the bin count to the noise, and prefer adaptive bins on wide-range targets,** where a fixed bin count is both less accurate and less stable.

## Appendix A. The law of total variance and the Gaussian log-likelihood

**Law of total variance.** Fix an input  $x$  and write  $Y$  for the (random) target there and  $B \in \{1, \dots, K\}$  for the bin it falls in, with  $\Pr(B = k) = \pi_k$ . The quantities  $\mu_k, \sigma_k^2, \pi_k$  below are the per-input values  $\mu_k(x), \sigma_k^2(x), \pi_k(x)$  of Section 2.1 with the argument suppressed. Within bin  $k$  the target has mean  $\mu_k$  and variance  $\sigma_k^2$ , and the overall mean is  $\hat{y} = \mathbb{E}[Y] = \sum_{k=1}^K \pi_k \mu_k$ .

The identity is derived from  $\text{Var}(Y) = \mathbb{E}[Y^2] - (\mathbb{E}[Y])^2$  by conditioning on the bin. Write  $\mathbb{E}[Y^2] = \mathbb{E}[\mathbb{E}[Y^2 | B]]$  and, within each bin,  $\mathbb{E}[Y^2 | B] = \text{Var}(Y | B) + (\mathbb{E}[Y | B])^2$ . Then

$$\text{Var}(Y) = \mathbb{E}[\text{Var}(Y | B)] + \left( \mathbb{E}[(\mathbb{E}[Y | B])^2] - (\mathbb{E}[\mathbb{E}[Y | B]])^2 \right) = \mathbb{E}[\text{Var}(Y | B)] + \text{Var}(\mathbb{E}[Y | B]), \quad (6)$$

where the bracketed difference is, by definition,  $\text{Var}(\mathbb{E}[Y | B])$ . Substituting the per-bin values, the first term is  $\mathbb{E}[\text{Var}(Y | B)] = \sum_{k=1}^K \pi_k \sigma_k^2$  (the within-bin variances averaged over the bin probabilities) and the second is  $\text{Var}(\mathbb{E}[Y | B]) = \sum_{k=1}^K \pi_k (\mu_k - \hat{y})^2$  (the variance of the bin means about the overall mean). Adding them gives the predictive variance used in Section 2.1. This is why the second term is read as “disagreement”: it is literally the variance of the bin means, and it is zero exactly when the means agree or all weight sits on one bin.

**Gaussian log-likelihood.** If the target is modelled as  $Y \sim \mathcal{N}(\hat{y}, \hat{\sigma}^2)$ , its density at the observed  $y$  is

$$p(y) = \frac{1}{\sqrt{2\pi\hat{\sigma}^2}} \exp\left(-\frac{(y - \hat{y})^2}{2\hat{\sigma}^2}\right). \quad (7)$$

The negative logarithm of this density is  $-\log p(y) = \frac{1}{2} \log(2\pi\hat{\sigma}^2) + \frac{(y - \hat{y})^2}{2\hat{\sigma}^2}$ , which is the per-example training loss of Section 3. Minimising its average over the data is maximum-likelihood estimation of the predictive Gaussian, and because the log-likelihood is a strictly proper scoring rule (Gneiting and Raftery 2007), its expected value is optimised only at the true conditional mean and variance.

## Appendix B. The gating penalty

---

The adaptive model’s extra term is a **Kullback–Leibler penalty** on the gate, added to the negative log-likelihood of Section 3. For a given input the gate produces a distribution  $q$  over the candidates of Section 2.3 — indexed by  $j$ : the bin counts  $2, \dots, K_{\max}$  and the bypass — and  $\rho$  is a fixed prior over those same candidates. The penalty is

$$\text{KL}(q \parallel \rho) = \sum_j q(j) \log \frac{q(j)}{\rho(j)}, \quad (8)$$

the average extra surprise from using the gate’s weighting  $q$  in place of  $\rho$ ; it is zero when  $q = \rho$  and grows as the gate concentrates away from the prior. The full per-example objective is the negative log-likelihood of the model’s predictive Gaussian (Section 3) *plus* this penalty — the likelihood pulling the gate toward whatever resolution fits the point, the penalty holding it near the prior.

The prior is deliberately **neutral**, and that is the point. It factors into a coin-flip on whether to bypass — prior probability one-half — and, given no bypass, a *uniform* spread over the bin counts, and the KL is applied in the same two parts. So it expresses no preference for coarse over fine, or for binning over bypassing: left to itself it simply keeps the gate diffuse, and only the likelihood pulls it toward a definite resolution. (An earlier version used a prior that leaned toward fewer bins, but that pull suppressed the correct resolution on heavy-tailed targets and was removed.) Because the penalty is a smooth function of the gate’s weights, the choice of resolution is trained end to end, with no separate search over bin counts.

## Appendix C. The three proper scores

---

Three of the seven scores are named in Section 5 but are worth giving exactly, since they carry the note’s argument. All are built on the same predictive Gaussian  $\mathcal{N}(\hat{y}, \hat{\sigma}^2)$  — with  $\hat{\sigma} = \sqrt{\hat{\sigma}^2}$  its standard deviation — and the same ninety-percent interval as everything else; write  $\Phi$  and  $\varphi$  for the standard normal cumulative and density functions and  $z = 1.645$  for the value with  $\Phi(z) = 0.95$ .

**Interval score.** For the central  $(1 - \alpha)$  interval  $[L, U]$  — here  $\alpha = 0.1$ ,  $L = \hat{y} - z\hat{\sigma}$ ,  $U = \hat{y} + z\hat{\sigma}$  — and true value  $y$ ,

$$S_\alpha(L, U; y) = (U - L) + \frac{2}{\alpha} (L - y) \mathbf{1}\{y < L\} + \frac{2}{\alpha} (y - U) \mathbf{1}\{y > U\} \quad (9)$$

(Gneiting and Raftery 2007): the width  $U - L$ , plus a penalty proportional to how far outside the truth falls, weighted by  $2/\alpha$  (here twenty), the indicator  $\mathbf{1}\{\cdot\}$  being one when its condition holds and zero otherwise. Because a miss is expensive, an interval cannot lower this score by shrinking below the coverage its accuracy supports. It is reported averaged over the test set.

**Distribution score.** For the predictive Gaussian the continuous ranked probability score has the closed form

$$\text{CRPS}(\mathcal{N}(\hat{y}, \hat{\sigma}^2), y) = \hat{\sigma} \left[ s(2\Phi(s) - 1) + 2\varphi(s) - \frac{1}{\sqrt{\pi}} \right], \quad s = \frac{y - \hat{y}}{\hat{\sigma}} \quad (10)$$

(Gneiting and Raftery 2007) — the integrated squared gap between the predicted cumulative distribution and the step that rises at  $y$ , in the units of the target. It stays finite even when a model is confidently wrong, which is why it is the robust cross-check on the log-likelihood.

**Calibration error.** Using the probability integral transform, for each test point compute  $u = \Phi((y - \hat{y})/\hat{\sigma})$ , the predictive probability of a value at or below the truth; if the model is calibrated the  $u$  are uniform on  $[0, 1]$ . At each of ten equally spaced levels  $\tau_j = j/11$  ( $j = 1, \dots, 10$ ), let  $\hat{c}(\tau_j)$  be the fraction of test points with  $u \leq \tau_j$ ; the calibration error — an *expected calibration error*, hence the label ECE below — is the mean absolute gap

$$\text{ECE} = \frac{1}{10} \sum_{j=1}^{10} |\hat{c}(\tau_j) - \tau_j|. \quad (11)$$

## References

---

- Chen, Tianqi, and Carlos Guestrin. 2016. “XGBoost: A Scalable Tree Boosting System.” In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–94. San Francisco, CA, USA: ACM. <https://doi.org/10.1145/2939672.2939785>.
- Gneiting, Tilmann, and Adrian E. Raftery. 2007. “Strictly Proper Scoring Rules, Prediction, and Estimation.” *Journal of the American Statistical Association* 102 (477): 359–78. <https://doi.org/10.1198/016214506000001437>.
- Ke, Guolin, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree.” In *Advances in Neural Information Processing Systems* 30 (*NeurIPS* 2017), 3149–57.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. “CatBoost: Unbiased Boosting with Categorical Features.” In *Advances in Neural Information Processing Systems* 31 (*NeurIPS* 2018), 6639–49.